

数字图像处理实验报告

实验一：数字图像基础

1. 实验目的

- 掌握不同格式图像的读写、显示、采样量化等基本操作，了解数字图像的文件格式和存储结构。
- 掌握对指定二进制图像数据格式的解析与显示。
- 掌握窗宽窗位调整、视频读取与显示等附加图像处理技术。

2. 实验原理

- **常见图像格式 (JPG, BMP, PNG):** 这些是标准化的文件格式。文件内部除了像素数据外，还包含元数据和压缩信息。JPG使用有损压缩，文件小，适合照片。BMP和PNG使用无损压缩，保留原始数据，文件较大。这些格式通常由高级库，如OpenCV直接支持，隐藏了底层的复杂性。
- **自定义二进制格式 (.dat):** 本实验中的 `.dat` 是一种原始图像格式。其结构是透明的，由开发者定义，通常不包含压缩。本次实验的格式由三部分组成：
 1. **主文件头：** 存储图像的全局信息，如尺寸、深度、通道数。
 2. **辅助信息头：** 紧跟主文件头，数量与图像高度相同，每一行都有一个对应的辅助头，用于描述该行像素的特殊存储方式。
 3. **像素数据区：** 存储所有未压缩的原始像素数据。解析这种格式需要逐字节地读取并根据预定义的数据结构进行解释。
- **窗宽窗位：** 这是一种增强图像局部对比度的灰度映射技术。
 - **窗位：** 确定了重点观察的灰度中心。
 - **窗宽：** 确定了观察范围的宽度。
 - **映射过程：** 根据窗位和窗宽，计算出灰度区间的下限 $\text{min_val} = C - W/2$ 和上限 $\text{max_val} = C + W/2$ 。原始图像中，所有灰度值低于 `min_val` 的像素被置为，高于 `max_val` 的像素被置为255（白）。而位于 `[min_val, max_val]` 区间内的像素，则通过线性变换被拉伸映射到完整的 `[0, 255]` 显示范围。

3. 实验过程

本次实验综合实现了对标准图像格式和自定义 `.dat` 格式的处理，并完成了所有附加任务。

- **第一部分：常见图像格式的读写与转换**

- **读写：** 使用 `cv2.imread("lena.jpg")` 函数读取一张常见的JPG图像，并通过 `cv2.imshow()` 进行显示。
- **保存与转换：** 使用 `cv2.imwrite()` 函数，通过简单地改变输出文件的后缀名，将内存中的图像数据分别保存为 `lena_png.png` 和 `lena_bmp.bmp`，从而实现了图像格式的转换。

- **第二部分：自定义 .dat 格式的解析与生成**

- **解析 (`read_and_display_dat_file`):**

1. 以二进制只读模式(`'rb'`)打开 `.dat` 文件。
2. 使用 `struct.unpack()` 根据预定义的格式字符串 `'<I10sIIHH'` 解析主文件头，提取出图像的宽、高、深度等元数据。
3. 循环读取 `height` 次，每次解析一个辅助信息头，并将描述该行像素存储方式的 `storage_mode` 标志字节存入列表。
4. 一次性读取文件剩余的全部像素数据。
5. 再次循环 `height` 次，对每一行数据，根据之前存储的 `storage_mode` 标志位进行按位解析（位移和与操作），判断其存储顺序（左到右/右到左）、存储方式（逐像素/逐通道）和通道类型（BGR/RGB），并使用Numpy的切片和重塑操作，将原始数据流重组为正确的图像行。
6. 将所有处理好的行数据堆叠起来，形成最终的图像数组并显示。

- **生成 (`save_image_to_custom_dat`):**

1. 从输入的图像数组中获取元数据。
2. 使用 `struct.pack()` 将元数据和固定的文件标识等打包成二进制的主文件头。
3. 为图像的每一行创建并打包对应的辅助信息头。
4. 以二进制写模式(`'wb'`)创建文件，并依次将主文件头、所有辅助信息头以及图像像素数据的字节流写入文件。

- **第三部分：附加任务的实现**

- **窗宽窗位调整：**

1. **静态调整 (`adjust_window_level`):** 实现了一个接收固定窗宽窗位值的函数，对图像进行一次性处理并显示。
2. **交互式调整 (`image_interactive_adjust`):** 创建了一个OpenCV窗口和两个滑块。当用户拖动滑块时，触发回调函数，该函数实时获取滑块值，对原始

图像应用窗宽窗位变换并刷新显示。

- 视频读取与显示 (`video_display`):

1. 使用 `cv2.VideoCapture()` 打开视频文件。
2. 在逐帧播放的循环中，同样创建了两个滑块。
3. 每一帧都主动查询滑块的当前值，并对当前帧应用窗宽窗位变换，从而实现了视频流的动态实时调整。

4. 实验结果

- 常见格式读写与转换

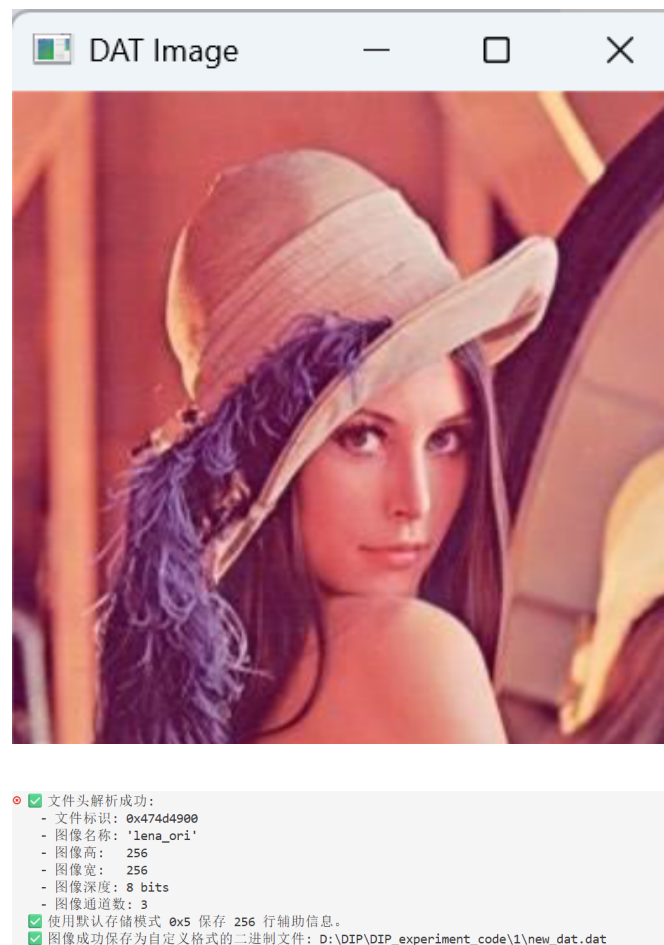


图1-1 二进制格式处理结果

- 自定义.dat格式解析



图1-2 自定义.dat格式重建结果

- 窗宽窗位调整与视频显示

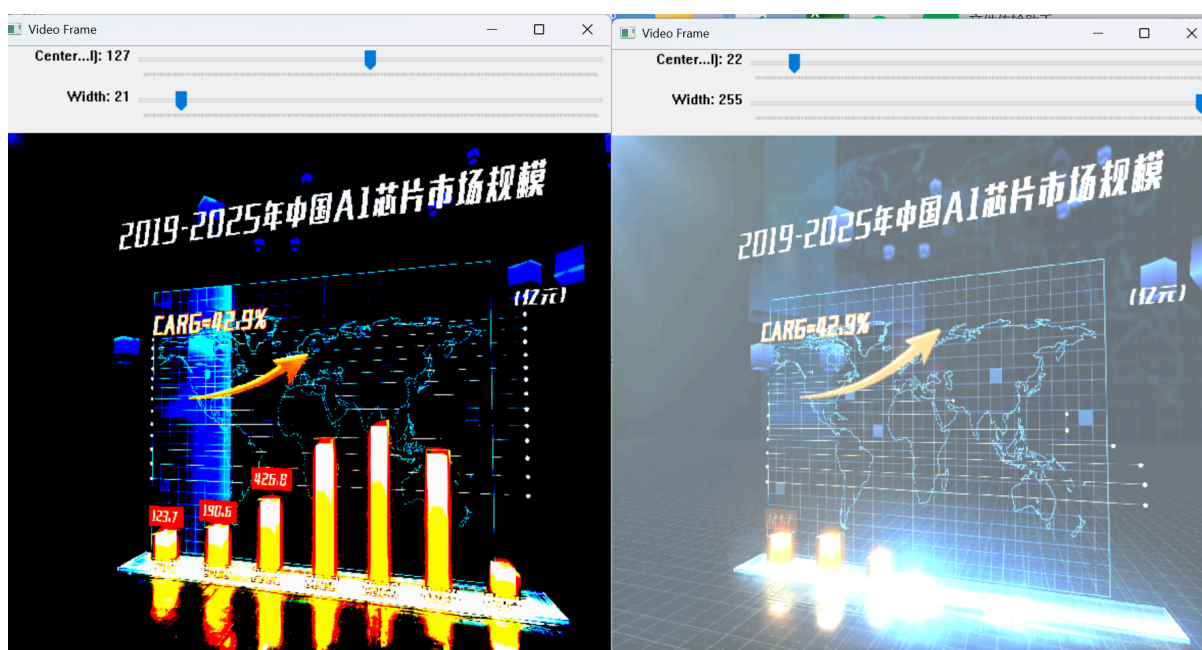


图1-3 交互式窗宽窗位调整及视频播放界面

5. 实验结论与分析

- 不同格式特点对比分析：

- **标准格式 (JPG, PNG, BMP):** 优点是高度标准化、通用性强，且有成熟的库支持，读写方便。缺点是内部结构被封装，用户无法直接控制。JPG通过有损压缩在文件大小上优势明显，适合网络传输；BMP结构简单，无损，但体积庞大；PNG是优秀的无损压缩格式，支持透明度，是Web和UI设计的首选。

- **自定义 .dat 格式:** 优点是结构完全透明，用户拥有最高控制权，可以根据特定需求定义存储方式（如本实验中的逐行变序），并且数据是无损的。缺点是缺乏通用性，需要编写专门的解析器，且无压缩导致存储效率低下。通过手动解析 .dat 文件，可以深刻理解所有图像文件“元数据头+像素数据块”的共通本质。
- **交互式处理的优越性:** 实验清晰地证明了交互式处理（通过滑块实时调整）相比于静态处理的巨大优势。对于窗宽窗位调整这类需要精细调参的任务，静态方法需要用户凭经验猜测参数，效率低下。而交互式方法提供了即时的视觉反馈，用户可以快速、直观地探索并定位到最能突显目标特征的参数组合，极大地提升了分析效率和体验。
- **算法的实时性与扩展性:** 窗宽窗位调整算法本身计算量很小（主要是剪裁和线性变换），实验证明其完全可以被实时地应用到视频流的每一帧上。这展示了将静态图像处理算法扩展到时域信号（视频）的可行性，为动态影像的实时增强分析奠定了基础。

6. 实验感想

- **遇到的问题:** 在解析 .dat 文件时，核心挑战在于正确解码 storage_mode 这个单字节标志位。最初，我对如何用一个字节表示三种不同的状态（存储顺序、存储方式、通道类型）感到困惑，并且对Python中的位运算操作不熟练，导致像素重组逻辑混乱。
- **解决方式:** 我通过查阅资料，复习了位掩码（mask）和位移（shift）操作。我编写了小的测试脚本，例如 `order = (byte >> 6) & 0x03`，来验证我提取各个标志位的逻辑是否正确。通过这种“分而治之”的调试方法，我最终成功地实现了对复杂存储模式的正确判断和像素数据的精确重组。
- **结果和收获:** 本次实验我成功地实现了对标准图像格式的转换、自定义二进制格式的完整解析与生成，并构建了一个功能完善的交互式图像/视频查看器。最大的收获是真正从底层字节层面理解了数字图像的存储方式，并掌握了如何通过位运算来高效地处理紧凑的标志位数据。这不仅是一次编程练习，更是对计算机数据表示和文件I/O原理的一次深刻实践。将理论算法与GUI编程结合，让我完成了从“知道一个算法”到“制作一个可用工具”的跨越，极大地提升了我对图像处理应用设计的理解。

实验二：图像变换

1. 实验目的

- 掌握图像变换的基本原理。
- 了解数字图像在空间域和变换域（频率域）的不同特点。

2. 实验原理

图像变换是将图像从其标准的空间域表示转换到变换域中。

- **傅里叶变换：** 傅里叶变换将图像分解为不同频率和振幅的正弦波和余弦波的叠加。变换后的结果即为图像的频谱。在频率域中，图像的低频分量对应图像中平缓、大面积的背景区域；高频分量对应图像的边缘、细节和噪声。
- **傅里叶变换性质：**
 - **平移性：** 图像在空间域的平移，其傅里叶变换的幅度谱不变，仅相位谱发生线性变化。
 - **旋转性：** 图像在空间域旋转一定的角度，其傅里叶变换的频谱图也跟着旋转相同的角度。
- **离散余弦变换 (DCT)：** DCT与FFT类似，也是一种频谱分析工具，但它只使用余弦函数作为基函数，变换结果是实数。

3. 实验过程

- **第一部分：傅里叶正反变换。**
 - **正向变换与频谱可视化：** 读取 `lena.jpg` 并转换为灰度图，再将其数据类型转为 `float32`。使用 `np.fft.fft2()` 计算二维快速傅里叶变换，得到一个复数矩阵。为了可视化，通过 `np.abs()` 计算其幅度谱，并使用 `np.log(1 + np.abs())` 进行对数变换以压缩动态范围，使得低幅度的频率分量也能被观察到。最后，调用 `np.fft.fftshift()` 将频谱图中的零频分量从左上角移到频谱中心，实现中心变换。
 - **逆向变换：** 对中心化后的频谱应用 `np.fft.ifftshift()`，先将其移回左上角。然后使用 `np.fft.ifft2()` 进行傅里叶逆变换。由于计算精度问题，逆变换结果仍是复数，通过 `np.abs()` 取其模得到实数图像数据，最终恢复出原始图像。通过 `cv2.absdiff()` 计算原图与恢复图像的差异，验证变换的可逆性。
- **第二部分：观察变换域性质。**
 - **平移：**
 1. **普通平移：** 使用 `cv2.warpAffine()` 对原图进行空间平移。计算平移后图像的频谱图，观察到其频谱图和原始图像的频谱图有一定的偏差，简单分析可以知道，是因为普通平移会对图像进行截断并对空缺部分进行补0处理，导致原始图像信息缺失。
 2. **循环平移：** 使用 `np.roll()` 实现图像的循环平移，同样计算其频谱图进行观察，观察到其幅度谱与原图基本一致，验证了傅里叶变换的平移不变性（幅度谱层面）。

- **旋转：** 使用 `cv2.getRotationMatrix2D()` 生成旋转矩阵，再通过 `cv2.warpAffine()` 对原图进行旋转。计算旋转后图像的频谱图，可以直观地看到频谱图也发生了与原图相同的旋转。
 - **附加任务：离散余弦变换（DCT）。**
 - **正反变换：** 使用 `cv2.dct()` 对 `float32` 类型的灰度图进行离散余弦变换。变换结果是一个实数矩阵，因此相较于傅里叶变换，省去了求频谱图绝对值的操作。
- 然后，使用 `cv2.idct()` 进行逆变换，将图像从DCT域恢复到空间域，同样由于结果为实数所以不需要取模操作。同时，计算恢复图像与原图的差异，以对比FFT和DCT的重建精度。

4. 实验结果

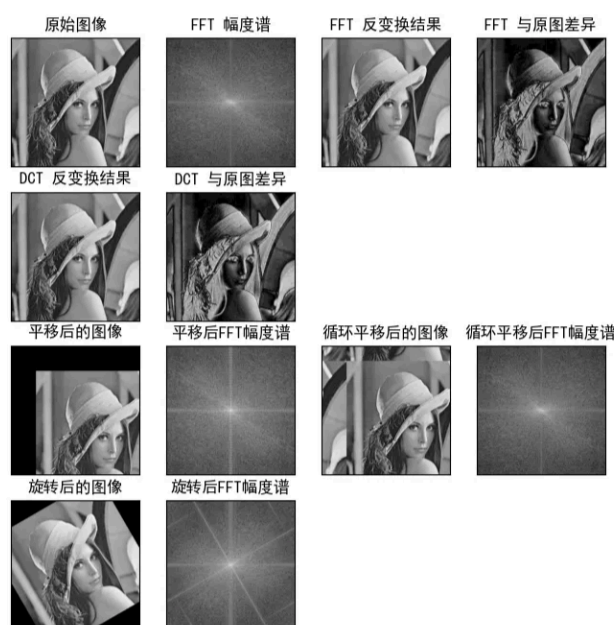


图2 图像变换实验结果汇总

5. 实验结论与分析

- **频谱图分析：** 傅里叶变换成功地将图像从空间域转换到了频率域。频谱图中心区域的亮点表明图像的大部分能量集中在低频分量，这与自然图像中通常包含大面积平缓区域的特性相符。远离中心的、较暗的区域则代表了构成图像边缘和细节的高频分量。对数变换是可视化频谱的关键，否则高频部分的细节将完全无法观察。
- **变换性质验证：**
 - **平移性：** 实验结果清晰地展示了普通平移和循环平移的区别。对于普通平移，由于移出边界的图像部分被丢弃，空白区域被补零，这相当于给图像乘以

了一个窗函数，引入了新的高频分量，因此其幅度谱出现了十字交叉状的干扰纹路，与原图幅度谱不同。而循环平移则保证了图像信息的完整性，其变换完全符合傅里叶变换的位移定理，因此其幅度谱与原图幅度谱几乎完全一致，仅相位谱发生了变化。

- **旋转性：** 从“旋转后的图像”及其对应的“旋转后FFT幅度谱”可以看出，当空间域的图像旋转一定角度后，其频率域的幅度谱也相应地旋转了完全相同的角度，这直观地验证了傅里叶变换的旋转特性。

- **FFT与DCT对比分析：** 从反变换结果与原图的差异图像来看，FFT和DCT都表现出极高的重建精度，证明了它们在理论上的可逆性。

DCT的反变换结果与原图差异甚至比FFT更小，这部分得益于DCT变换结果为实数，避免了FFT复数运算中可能引入的微小精度误差。

需要补充的是，DCT具有出色的“能量集中”特性，即用更少的系数就能表示图像的主要信息，这也是它被JPEG压缩标准选为核心算法的根本原因。

6. 实验感想

- **遇到的问题：** 最初进行傅里叶逆变换后，得到的图像一片漆黑或完全错误。另一个问题是，直接显示 `np.abs(f)` 得到的频谱图，只能看到中心一个白点，周围都是黑的。
- **解决方式：** 经过调试发现，逆变换后的数据是 `float` 类型，且范围不一定在 `[0, 255]`，需要进行归一化并转换为 `uint8` 才能正确显示。对于频谱图显示问题，通过查阅资料学习到，必须进行对数变换 `np.log(1 + np.abs())` 来压缩频谱的动态范围，才能同时观察到低频和低频分量。
- **结果和收获：** 本次实验让我对图像的频率域表示有了非常直观和深刻的认识。通过在空间域对图像进行平移和旋转，并观察其在频率域的对应变化的，我掌握了傅里叶变换的重要性质。

实验三：图像增强

1. 实验目的

- 掌握图像直方图统计、灰度变换、直方图均衡等基本图像增强原理和方法。
- 掌握常用的平滑去噪滤波方法。
- 通过对比不同处理方法，了解图像增强的实际效果。

2. 实验原理

图像增强旨在通过特定的算法处理，有选择性地突出图像中感兴趣的特征，或抑制不需要的特征，使其更适合于人类观察或后续的机器分析。

- **直方图均衡化：**通过变换函数重新分布图像的灰度级，使得输出图像的直方图尽可能平坦，从而扩展了动态范围，增强了全局对比度。
- **空间滤波：**通过滤波器与图像进行卷积。

均值滤波对邻域像素取平均，能有效平滑高斯噪声，但会模糊边缘。

中值滤波取邻域像素的中值，对椒盐噪声有特效，且保边性能优于均值滤波。

3. 实验过程

- **第一部分：基于直方图的对比度增强。**
 - **直方图统计与显示：**读取图像并转为灰度图。将二维图像数组通过 `.ravel()` 展平为一维，然后调用 `matplotlib.pyplot.hist()` 函数直接计算并绘制出其灰度直方图。
 - **全局线性灰度拉伸：**使用 `cv2.normalize()` 函数，并指定归一化类型为 `cv2.NORM_MINMAX`。该函数会自动查找图像的最小和最大灰度值，并将这个范围线性地拉伸到 `[0, 255]`，从而实现对比度的增强。
 - **直方图均衡化：**使用 `cv2.equalizeHist()` 函数。该函数会自动计算图像的累积分布函数并以此为映射，将原图像的灰度分布变得更加均匀，从而达到增强全局对比度的目的。
- **第二部分：噪声添加与平滑去噪。**
 - **噪声生成：**
 1. **高斯噪声：**使用 `np.random.normal()` 生成一个与原图尺寸相同、符合高斯分布（均值为0，标准差为25）的噪声矩阵，并将其直接加在原图上。最后使用 `np.clip()` 将结果像素值限制在 `[0, 255]` 范围内。
 2. **椒盐噪声：**首先根据噪声比例 `amount` 计算出需要添加的“盐”点（白色）和“椒”点（黑色）的数量。然后，使用 `np.random.randint()` 生成这些点的随机坐标，并将对应位置的像素值分别设为255和0。
 - **平滑滤波：**
 1. **均值滤波：**调用 `cv2.blur()` 函数，并传入一个 `(kernel_size, kernel_size)` 的核尺寸（如5x5），对噪声图像进行平滑处理。
 2. **中值滤波：**调用 `cv2.medianBlur()` 函数，并传入一个核尺寸（如5），对噪声图像进行滤波。
 - **效果对比：**将两种滤波器分别应用于两种噪声图像，通过对比处理结果，可以直观地分析不同滤波器对不同类型噪声的抑制效果和对图像细节的保护能力。

4. 实验结果

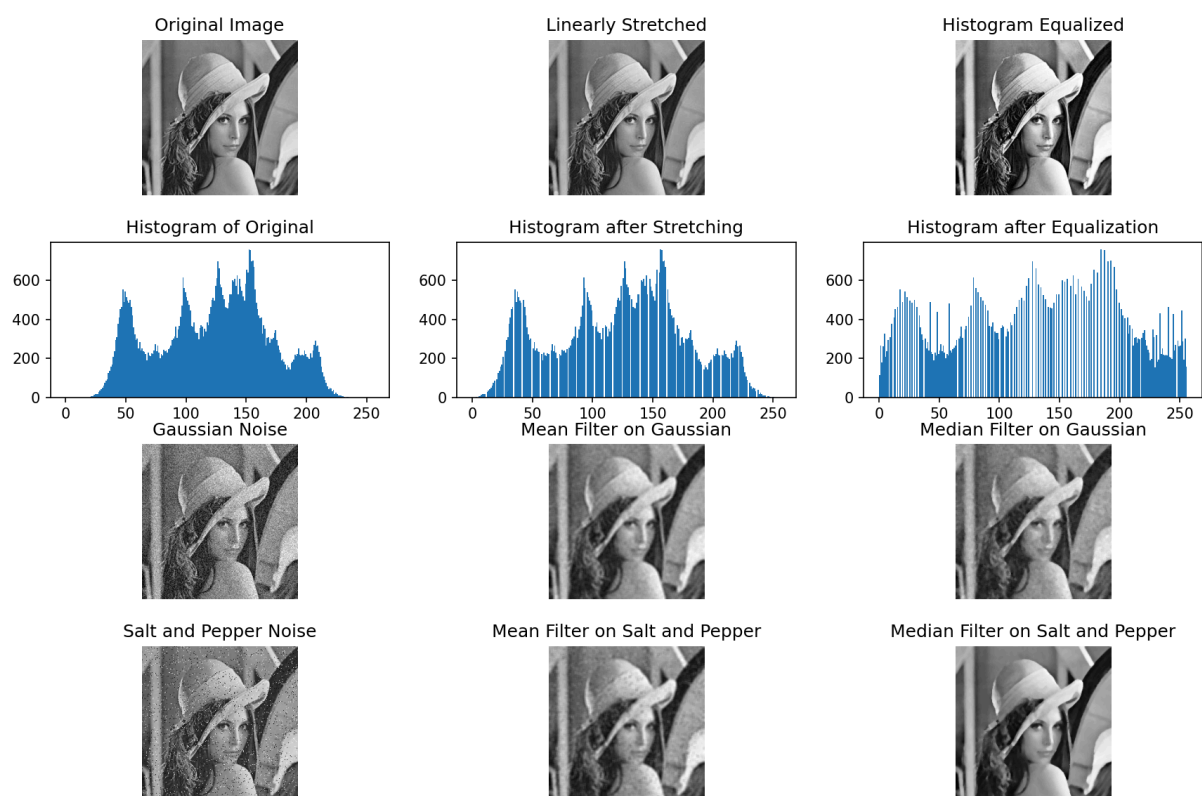


图3 图像增强与去噪实验结果汇总

5. 实验结论与分析

- 对比度增强分析：** 从结果图第一行可以看出，原始图像的直方图灰度级主要集中在50-200之间，导致整体视觉效果偏灰，对比度不足。“线性拉伸”通过将此范围扩展到整个 $[0, 255]$ 区间，显著改善了对比度，其直方图形态被“拉宽”了，但原有峰谷结构和空隙依然存在。“直方图均衡化”则更进一步，它进行非线性映射，使得最终的直方图分布非常均匀平坦，图像的动态范围得到了最大化利用，暗部和亮部的细节都得到了很好的展现，增强效果最为显著。
- 去噪滤波器对比分析：**
 - 对于高斯噪声（第三行）：** 均值滤波表现更佳。因为高斯噪声的值在均值附近随机波动，通过邻域平均可以有效地抵消这些波动，使得图像变得平滑。然而，其代价是图像的边缘也被平均化，导致了明显的模糊。中值滤波也能去除部分高斯噪声，但效果不如均值滤波，保留了更多的高频噪声。
 - 对于椒盐噪声（第四行）：** 中值滤波效果远胜于均值滤波。椒盐噪声是幅值极大或极小的离群点，均值滤波会受到这些极端值的严重影响，将一个黑/白噪点“平均”成一个灰色暗/亮斑，效果很差。而中值滤波作为一种非线性排序统计滤波器，能够有效地忽略这些极端离群值，因此可以近乎完美地去除椒盐噪声，同时由于它保留了邻域中的真实像素值，其保边性能也更好，图像清晰度受损很小。

6. 实验感想

- **遇到的问题：** 在实验过程中，虽然理论上理解均值滤波和中值滤波的差异，但在观察对高斯噪声的处理结果时，中值滤波似乎也展现了一定的平滑能力，这让我对其与均值滤波在处理高斯噪声时的性能优劣产生了进一步的思考。
- **解决方式：** 对于滤波性能的思考，我意识到均值滤波作为一种线性操作，其本质是低通滤波，对高斯白噪声这种在频域均匀分布的噪声有理想的抑制效果。而中值滤波作为非线性排序统计滤波器，虽然能滤除噪声，但它对噪声统计分布的假设与高斯分布并不完全匹配，因此效果不如均值滤波。
- **结果和收获：** 本次实验成功地验证了直方图均衡化在改善图像动态范围方面的有效性，并直观地对比了两种经典空间滤波器在不同噪声模型下的性能。最大的收获在于将理论知识与实践结果进行了深度印证。例如，实验结果清晰地展示了中值滤波对椒盐噪声这种脉冲型噪声近乎完美的滤除能力，这正是其非线性、基于排序特性的体现。而均值滤波在处理椒盐噪声时产生的“晕开”效应，则暴露了其作为线性滤波器对离群点敏感的固有缺陷。

实验四：图像锐化

1. 实验目的

- 掌握图像锐化的基本原理和方法。
- 掌握彩色增强的基本原理和方法。

2. 实验原理

图像锐化是一种与平滑相反的操作，其目的是增强图像的边缘和细节，使模糊的图像变得清晰。

- **空间域锐化：** 图像的边缘和细节对应于灰度值的突变，在数学上可以用微分来检测。
- **HSI颜色空间：** 人类对颜色的感知主要基于色调（Hue）、饱和度（Saturation）和亮度（Intensity）。HSI颜色模型将这三个分量分离开来，使得可以独立地调整它们。通过转换到HSV空间，我们可以单独增强S（饱和度）通道使颜色更鲜艳，或增强V（亮度）通道来调整明暗对比，而不会相互干扰。
- **伪彩色增强：** 这是一种将灰度图像转换为彩色图像的技术。其原理是根据原图中像素的灰度值，将其映射到一个预定义的颜色表中。

3. 实验过程

- **第一部分：高通模板锐化**

- **准备模糊图像：** 首先，使用 `cv2.GaussianBlur()` 对原始灰度图像进行高斯模糊处理，以模拟一张需要锐化的图像。

- **定义锐化核：** 代码中定义了两个不同的拉普拉斯锐化核。

- 标准锐化核：

```
[[0, -1, 0]
```

```
[-1, 5, -1]
```

```
[0, -1, 0]。
```

- 加权锐化核：

```
[-1, -1, -1]
```

```
[-1, 9, -1]
```

```
[-1, -1, -1]
```

由于其周围负值的权重更大，中心正值也更大，因此锐化效果会更强烈。

- **应用卷积：** 使用 `cv2.filter2D()` 函数，将这两个卷积核分别应用于模糊图像，得到两种不同强度的锐化结果。

- **第二部分：HSI空间彩色增强**

- **空间转换：** 使用 `cv2.cvtColor()` 函数将原始BGR彩色图像转换到HSV颜色空间。

- **通道分离与增强：** 使用 `cv2.split()` 将HSV图像分离成H、S、V三个独立的单通道图像。然后，分别对H、S、V通道的像素值乘以一个增强因子。

```
Hue_factor = 2.0 saturation_factor = 1.9 Intensity_factor = 1.8
```

- **通道合并与转回：** 使用 `cv2.merge()` 将增强后的通道与另外两个未修改的通道合并回一个HSI图像。再次调用 `cv2.cvtColor()` 将其转换回RGB空间，以便于 `Matplotlib` 正确显示。

- **附加任务：伪彩色增强**

- **手动映射：** 首先创建一个与原灰度图尺寸相同的全黑彩色图像。然后，根据预设的灰度阈值向量 `[50, 100, 150, 200]`，使用Numpy的布尔索引功能，将不同灰度区的像素批量地赋予指定的颜色，在本次实验中分别对应黑色，红色，绿色，蓝色，白色。

4. 实验结果

图像锐化与彩色增强实验结果

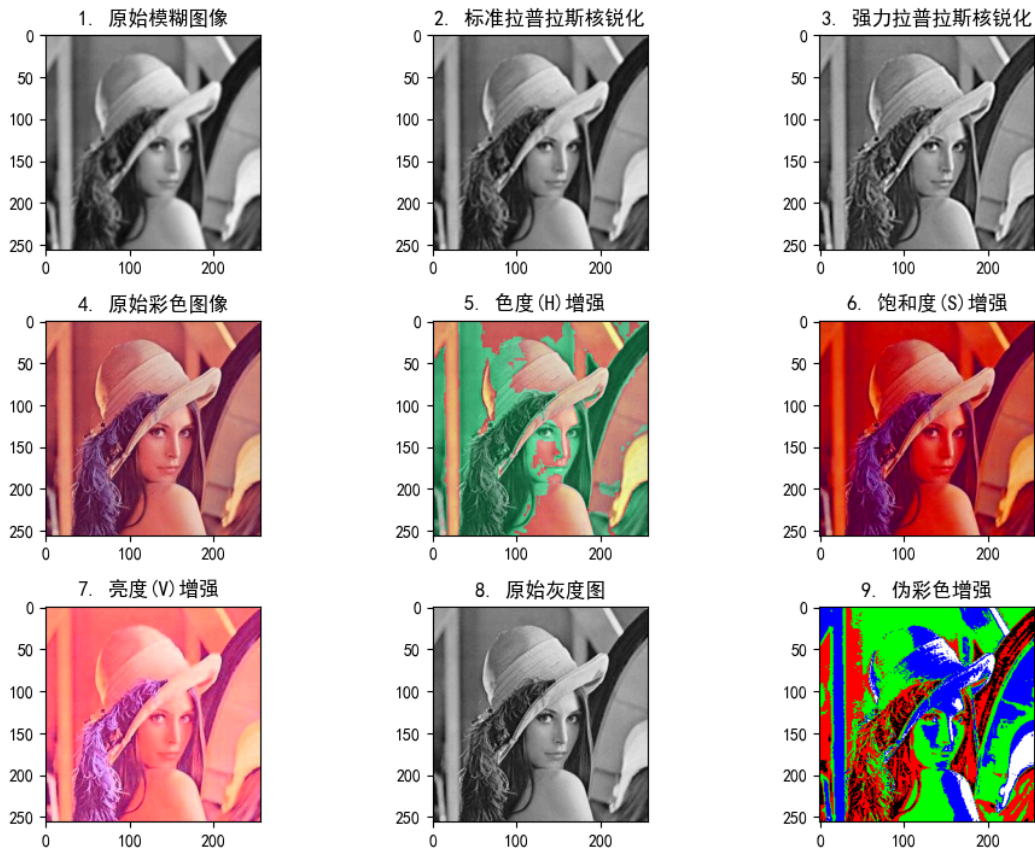


图4 图像锐化与彩色增强实验结果

5. 实验结论与分析

- **高通模板锐化分析：** 从结果图第一行可以看出，通过卷积应用拉普拉斯核可以有效地锐化模糊图像，使帽子和面部的边缘、细节更加突出。对比“标准拉普拉斯核锐化”和“强力拉普拉斯核锐化”可以发现，第二个核由于中心权重更高，周围负权重范围更广，其锐化效果更强，边缘对比度更高，但也使得图像的颗粒感更强，这说明锐化强度的提升往往伴随着对潜在噪声的同步放大。
- **HSI空间增强分析：** HSI颜色空间在彩色增强方面显示出巨大优势。结果图清晰地展示了对不同通道操作的效果：增强H通道导致了剧烈的颜色变化，人物皮肤呈现绿色，说明色度决定了颜色的基本类别；增强S通道使得图像色彩变得异常鲜艳浓郁，但并未改变颜色本身；增强V通道则使整个图像的明暗度提升，类似于增加了曝光。这种将色度与亮度分离的处理方式，其结果比直接在RGB空间操作更加直观和可控。
- **伪彩色增强分析：** 伪彩色技术是一种强大的可视化工具，从结果图的右下角可以看出，它成功地将单调的灰度图像转换为了信息丰富的彩色图像。原本难以区分

的细微灰度差异，如面部光影，被映射成了非常敏感的颜色差异，使得图像中不同灰度等级的区域分布变得一目了然，极大地提高了图像信息的可读性。

6. 实验感想

- **遇到的问题：** Matplotlib中显示OpenCV读取的彩色图像时，颜色总是错的，具体而言是蓝色和红色相互颠倒。
- **解决方式：** 对于颜色显示问题，我查阅资料后了解到OpenCV默认的颜色通道顺序是BGR，而Matplotlib是RGB，因此在显示前必须使用 `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` 进行转换。
- **结果和收获：** 本次实验让我掌握了多种使图像信息“更清晰”的技术。我不仅学会了如何通过空间滤波来锐化图像，更重要的是理解了HSI颜色模型在彩色图像处理中的优越性。伪彩色增强的实践则让我认识到颜色可以作为一种强大的信息编码工具。这次实验让我深刻体会到，图像处理不仅仅是算法的堆砌，更是要结合人类视觉系统的特性，才能做出真正有效的增强。

实验五：图像复原

1. 实验目的

- 掌握图像模糊复原的基本原理和方法，了解图像复原的效果。

2. 实验原理

图像复原试图根据图像的退化模型，将退化的图像恢复到其原始状态。这与主观的图像增强不同，复原是一个客观的过程。

- **图像退化模型：** 一个普遍的退化模型可以表示为 $g(x, y) = h(x, y) * f(x, y) + n(x, y)$ ，其中 `g` 是退化图像，`f` 是原始图像，`h` 是点扩散函数，`n` 是噪声。

复原的目标就是从 `g` 和对 `h`、`n` 的了解中，估计出 `f`。

- **点扩散函数：** PSF描述了退化系统的响应。运动模糊的PSF是一条线段，其长度和角度定义了模糊。离焦模糊的PSF是一个圆盘，其半径决定了模糊程度。
- **维纳滤波：** 维纳滤波是一种基于最小均方误差准则的经典复原方法。它在频率域进行操作，其公式为 $F'(u, v) = [1/H(u, v)] * [|H(u, v)|^2 / (|H(u, v)|^2 + K)] * G(u, v)$ 。其中 `H` 和 `G` 分别是退化函数和退化图像的傅里叶变换，`K` 是一个与信噪比有关的常数。维纳滤波能够在反卷积和抑制噪声之间取得一个理想的平衡。

3. 实验过程

- **第一部分：点扩散函数的生成**

- **匀速直线运动PSF**：通过在零矩阵中心使用 `cv2.line()` 绘制一条指定长度和角度的线段来创建PSF。
- **任意运动PSF**：通过 `cv2.polylines()` 连接一系列预设点，模拟更复杂的运动轨迹，在本次实验中我假定的运动为Z字形运动。
- **离焦PSF**：使用 `cv2.circle()` 在零矩阵中心绘制一个指定半径的实心圆盘。
- 所有生成的PSF最后都进行归一化，使其元素总和为1。
- **第二部分：模拟图像退化**
 - **模糊**：使用 `cv2.filter2D()` 将原始清晰图像与上一步生成的各种PSF进行卷积，得到对应的模糊图像。
 - **加噪**：对于维纳滤波实验，使用 `np.random.normal()` 生成高斯噪声，并将其添加到模糊图像上，以模拟更真实的退化场景。
- **第三部分：维纳滤波复原**
 - **频域转换**：首先将PSF填充到与模糊图像相同的大小，并将其中心对齐。然后，使用 `np.fft.fft2()` 分别计算模糊图像和中心化PSF的傅里叶变换，得到 $G(u,v)$ 和 $H(u,v)$ 。
 - **应用维纳滤波器**：根据维纳滤波的频域公式，计算出滤波器的传递函数 $W(u,v) = H^* / (|H|^2 + K)$ 。其中 K 是需要手动调整的关键参数。
 - **频域复原**：将模糊图像的频谱 $G(u,v)$ 与维纳滤波器 $W(u,v)$ 相乘，得到复原图像的频谱 $F'(u,v)$ 。
 - **空间域转换**：使用 `np.fft.ifft2()` 对 $F'(u,v)$ 进行傅里叶逆变换。取结果的实部，并通过 `np.fft.fftshift()` 将图像内容移回中心，最后归一化到 $[0, 255]$ 范围，得到最终的复原图像。

4. 实验结果



图5 图像复原实验结果汇总

5. 实验结论与分析

- 复原效果总体分析：** 实验结果清晰地展示了图像复原的强大能力。对于各种类型的模糊，只要其退化过程是已知的，维纳滤波都能够显著地恢复出图像的细节。无论是“高斯模糊”、“匀速运动模糊”还是“离焦模糊”，复原后的图像清晰度都得到了极大提升。这与实验3的图像增强有本质区别，复原是基于数学模型的“逆运算”，目标是逼近原始图像。
- 噪声与K值的影响：** 对比“高斯模糊+噪声”和“高斯模糊+没有噪声”两组实验，可以发现维纳滤波的核心优势在于其对噪声的鲁棒性。在有噪声的情况下，复原图像虽然不如无噪声情况清晰，但依然有效地抑制了噪声的过度放大，避免了纯粹逆滤波会导致的噪声溢出。这说明参数 K 起到了关键的正则化作用，在去模糊和抑制噪声之间取得了平衡。
- PSF的重要性与复杂性：** 实验中对不同运动轨迹的复原结果表明，PSF的复杂性直接影响复原难度。对于简单的匀速运动，复原效果很好，基本消除了拖影。而对于更复杂的“Z字形运动模糊”，虽然复原后图像的可辨识度有所提高，但仍存在明显的振铃效应和伪影，说明对复杂退化过程的逆运算更加困难。这凸显了在实际应用中，准确地估计PSF是图像复原成功的关键和挑战。

6. 实验感想

- **遇到的问题：** 在实现维纳滤波时，最大的挑战在于如何为不同的退化场景设定一个合适的超参数 K 。最初，我只能通过大量盲目试错来调整 K 值，微小的变动都会导致结果在“去模糊不足”和“噪声被灾难性放大”之间剧烈摆动，缺乏系统性的调优方法。
- **解决方式：** 为了解决这一问题，我深入研究了维纳滤波的理论推导。我认识到 K 本质上是噪声功率谱与原始信号功率谱之比的估计，即信噪比的倒数。这一物理意义为调参提供了明确指引：对于噪声较小、退化函数信噪比较高的场景，如纯运动模糊， K 值应非常小以最大化去模糊效果；而对于噪声显著的场景，则需要增大 K 值以牺牲部分细节恢复为代价，来换取对噪声的有效抑制。
- **结果和收获：** 本次实验让我对图像复原作为一种不适定反问题有了深刻的实践认知。最大的收获是明白了，即使在退化模型完全已知的“理想”条件下，求解一个完美的逆过程依然充满挑战。维纳滤波作为一种经典的线性最小均方误差解，提供了一个优雅的统计学框架来平衡解的稳定性和保真度。这次经历也让我清晰地看到了该领域的两大核心难点：一是如何从退化图像中精确估计PSF和噪声统计特性，二是如何设计出能更好地保留边缘、抑制伪影的高级正则化项或非线性复原算法。

实验六：图像分割

1. 实验目的

- 掌握阈值分割、边缘检测等图像分割的基本原理和方法。
- 掌握Hough变换检测直线和圆的原理与应用。

2. 实验原理

图像分割是将数字图像划分为多个互不相交的区域的过程，使得同一区域内的像素具有相似的属性。

- **阈值分割：** 通过设定一个或多个灰度阈值，将像素分为前景和背景。对于光照不均的图像，自适应阈值更为有效，它不是使用全局唯一的阈值，而是为每个像素根据其邻域的特征（如均值或高斯加权和）动态地计算一个阈值。
- **边缘检测：** 分割也可以通过寻找区域之间的边界来实现。
 - **Sobel算子**是一阶微分算子，计算梯度来检测边缘，结果较粗。
 - **LoG算子**是二阶微分算子，它先对图像进行高斯平滑以抑制噪声，再用拉普拉斯算子检测边缘，对噪声敏感度较低。

- **Canny算子**是一种多阶段的最优边缘检测算法，包括高斯滤波、非极大值抑制和双阈值迟滞连接，能产生精细、连续的单像素边缘。
- **Hough变换**：这是一种用于检测图像中特定形状（如直线、圆）的特征提取技术。其思想是将图像空间中的特征点映射到参数空间进行“投票”，参数空间中的峰值即对应了原始图像中存在的特定形状。

3. 实验过程

• 第一部分：自适应阈值分割

- **过程**：调用 `cv2.adaptiveThreshold()` 函数。代码中对比了两种方法：
`cv2.ADAPTIVE_THRESH_MEAN_C`（邻域块的均值）和 `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`（邻域块的高斯加权）。`blockSize` 定义了邻域的大小，`C` 是一个从均值或加权和中减去的常数，用于微调。同时，使用大津法（`cv2.THRESH_OTSU`）作为全局阈值分割的对比。

• 第二部分：边缘检测算子比较

- **Sobel算子**：使用 `cv2.Sobel()` 分别计算x和y方向的梯度，然后求其平方和的平方根来得到梯度幅值，最后归一化显示。
- **LoG算子**：先用 `cv2.GaussianBlur()` 对图像进行平滑处理，然后用 `cv2.Laplacian()` 计算二阶导数，最后取绝对值得到边缘图。
- **Canny算子**：直接调用 `cv2.Canny()` 函数，并设定高低两个阈值。

• 第三部分：Hough变换检测直线与圆

- **准备测试图像**：为了清晰地展示效果，代码中创建了一张包含多条直线和多个空心圆的黑色背景测试图。
- **直线检测**：
 1. 首先使用 `cv2.Canny()` 提取边缘。
 2. 然后将边缘图输入 `cv2.HoughLines()` 函数。该函数返回检测到的直线的极坐标参数 `(rho, theta)`。
 3. 最后，将这些参数转换回笛卡尔坐标，并在原图的副本上用 `cv2.line()` 绘制出检测到的红色直线。
- **圆检测**：
 1. 由于圆检测对噪声敏感，先使用 `cv2.medianBlur()` 对灰度图进行中值滤波。
 2. 调用 `cv2.HoughCircles()` 函数。该函数内部集成了边缘检测，`dp` 是累加器分辨率，`minDist` 是圆心之间的最小距离，`param1` 和 `param2` 是Canny边缘检测和

圆心检测的阈值。

3. 遍历检测到的所有圆，并在图像上用 `cv2.circle()` 绘制出绿色的圆边界和蓝色的圆心。

4. 实验结果

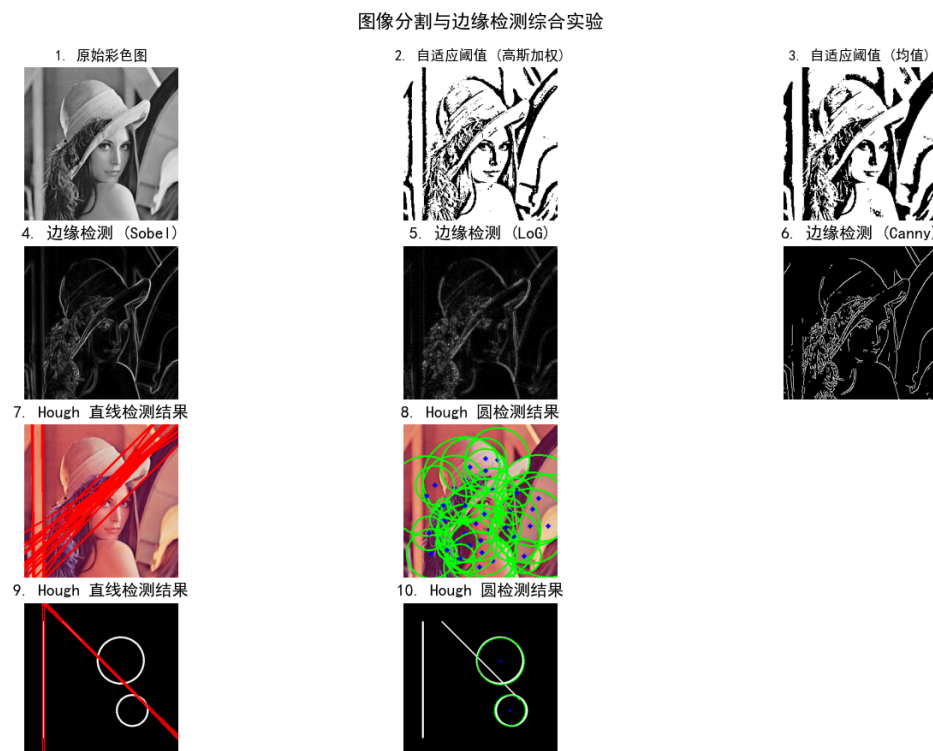


图6 图像分割与特征检测实验结果

5. 实验结论与分析

- **阈值分割分析：** 对于具有复杂光照和纹理的Lena图像，自适应阈值分割的效果（图2、3）明显优于全局阈值。它能够根据局部区域的亮度特性动态调整阈值，从而在较亮的脸部和较暗的背景中都能有效地分离出轮廓。对比“高斯加权”和“均值”两种方法，高斯加权的结果通常更平滑，对噪声的抑制更好。
- **边缘检测算子对比分析：**
 - **Sobel算子**（图4）作为一阶微分算子，能检测出大部分强边缘，但其结果边缘较粗，且容易产生双边效应，丢失了部分细节。
 - **LoG算子**（图5）是二阶微分算子，对细线和细节更敏感，检测出的边缘更细，但同时也对噪声非常敏感，导致结果中包含大量由纹理产生的杂乱边缘。
 - **Canny算子**（图6）综合性能最优。它通过多阶段处理，其结果边缘是精细的单像素宽度，连续性好，并且有效地抑制了大部分噪声和伪边缘，清晰地勾勒

出了图像的主要轮廓。

- **Hough变换分析：** Hough变换展示了其作为全局特征检测器的强大能力。对比真实图像（图7、8）和测试图像（图9、10）的结果：
 - 在**测试图像**上，由于只存在完美的直线和圆，Hough变换表现完美，准确地检测出了所有预设的几何形状。
 - 在**真实图像Lena**上，Hough直线变换检测出了帽子和背景中的大量直线结构；而Hough圆变换则检测出了眼睛、瞳孔等圆形特征，但也因为图像的复杂纹理产生了许多假正例。这说明Hough变换对具有清晰几何形状的物体检测非常有效，但在应用于复杂自然场景时，需要精细的参数调整和预处理来提高准确性。

6. 实验感想

- **遇到的问题：** 在使用Hough圆检测时，最初对真实图像 `lena.jpg` 的效果很差，检测出很多错误的圆。另外，对Canny算子的两个阈值 `threshold1` 和 `threshold2` 的选择感到困惑，不知道如何设置。
- **解决方式：** 查阅文档发现，Hough圆检测对噪声和复杂纹理非常敏感。因此，我在调用 `cv2.HoughCircles` 之前，先对图像进行了一次中值滤波 `cv2.medianBlur`，显著减少了噪声干扰，检测效果得到了很大改善。对于Canny阈值，我通过多次尝试和观察，理解了低阈值用于连接边缘，高阈值用于确定强边缘的起点，通常建议高低阈值比在2:1到3:1之间，并根据图像对比度进行调整。
- **结果和收获：** 本次实验让我系统地掌握了从像素级分割到特征级分割再到形状级识别的一系列关键技术。我深刻理解了Canny算子为何被誉为边缘检测的金标准，因为它巧妙地解决了检测过程中的多个核心难题。Hough变换的“投票”思想也让我印象深刻，它提供了一种从局部特征推断全局结构的有效范式。